

Day 4: 日志监控 + 测试结构 + 基础设施收尾 — 详细教程

目录

- [一、今日目标与验收标准](#)
 - [二、Python logging 模块配置与日志分级](#)
 - [三、后端全局异常处理中间件](#)
 - [四、API 请求日志记录中间件](#)
 - [五、健康检查接口完善](#)
 - [六、前端错误监控与上报](#)
 - [七、Docker 容器日志查看与管理](#)
 - [八、pytest 测试框架搭建](#)
 - [九、API 接口测试编写](#)
 - [十、前端单元测试基础 \(Vitest\)](#)
 - [十一、前后端完整联调验证](#)
 - [十二、常见问题排查](#)
-

一、今日目标与验收标准

1.1 今日任务总览

Day 4 是**第一阶段（项目架构搭建）**的最后一天，主要完成日志监控体系、测试框架搭建、前后端联调验证三件事。通过今天的实训，项目将具备企业级的可观测性和质量保障能力。

时间段	内容	形式
上午 0.5h	Python logging 模块配置、日志分级、日志文件轮转	讲授 + 实操
上午 0.5h	后端全局异常处理中间件	实操
上午 0.5h	API 请求日志记录（请求/响应/耗时）	实操
上午 0.5h	前端错误监控与上报	实操
上午 0.5h	Docker 容器日志查看与管理	实操
上午 0.5h	健康检查接口 <code>/api/health</code> 完善	实操
下午 1h	pytest 测试框架搭建、测试目录结构	实操
下午 0.5h	API 接口测试编写（认证接口）	实操
下午 0.5h	前端单元测试基础（Vitest）	实操
下午 1h	前后端完整联调：登录 → 基础页面交互	实操

1.2 验收标准

- ☐ 日志系统正常工作，可按级别输出到文件，日志文件可自动轮转
- ☐ 全局异常处理中间件可捕获未处理异常并返回友好 JSON 错误
- ☐ API 请求日志中间件记录每次请求的方法、路径、状态码、耗时
- ☐ pytest 测试框架可运行，至少包含认证模块的基础测试用例
- ☐ 前端 Vitest 测试框架可运行，至少包含工具函数的单元测试
- ☐ 前后端联调通过：登录 → 主界面 → 各页面路由正常
- ☐ 健康检查接口 `/api/health` 返回各服务状态

1.3 Day 4 结束后的目录结构

```
rsod-agent-platform/
├── backend/
│   ├── main.py                # 更新：注册中间件和新路由
│   ├── requirements.txt       # 更新：添加 pytest 等测试依赖
│   ├── .env
│   ├── alembic.ini
│   └── alembic/
│
│   ├── app/
│   │   ├── config/
│   │   │   └── settings.py    # 更新：添加日志配置项
│   │   └── core/
│   │       ├── security.py    # 已有
│   │       └── logger.py      # 【新增】日志配置模块
```

└─ exceptions.py	# 【新增】全局异常处理
└─ middleware/	# 【新增目录】中间件
└─ ── __init__.py	
└─ ── request_logger.py	# 【新增】请求日志中间件
└─ api/	
└─ ── auth.py	# 已有
└─ ── health.py	# 【新增】健康检查路由（从 main.py 拆分）
└─ ... (其他已有模块)	
└─ logs/	# 【新增目录】日志文件存放
└─ ── .gitkeep	
└─ tests/	# 【新增目录】测试
└─ ── __init__.py	
└─ ── conftest.py	# pytest 全局 fixtures
└─ ── test_health.py	# 健康检查测试
└─ ── test_auth.py	# 认证接口测试
└─ frontend/	
└─ ── src/	
└─ ── ── utils/	
└─ ── ── ── request.js	# 更新：添加错误上报逻辑
└─ ── ── ── errorReporter.js	# 【新增】前端错误上报
└─ ── ── ... (其他已有模块)	
└─ ── tests/	# 【新增目录】前端测试
└─ ── ── setup.js	# Vitest 全局 setup
└─ ── ── ── utils/	
└─ ── ── ── ── request.test.js	# Axios 封装测试
└─ ── package.json	# 更新：添加 vitest 依赖
└─ ── vite.config.js	# 更新：添加 vitest 配置
└─ docker-compose.yml	# 已有
└─ docs/	

二、Python logging 模块配置与日志分级

2.1 为什么需要日志系统

在 Day 2 中，我们用 `print()` 输出信息。但在企业级项目中，`print()` 存在严重不足：

print() vs logging 对比	
特性	对比结果

日志分级	print 无法区分 INFO/WARNING/ERROR logging 支持 DEBUG/INFO/WARNING/ERROR
输出目标	print 只能输出到终端 logging 可同时输出到终端+文件+远程服务
日志格式	print 需要手动拼接时间、模块名 logging 自动附加时间戳、模块、行号
文件轮转	print 不支持 logging 支持按时间/大小自动切割日志文件
生产可控	print 无法在运行时调整输出级别 logging 可通过配置文件动态控制

2.2 日志级别说明

Python logging 提供 5 个标准级别，按严重程度递增：

级别	数值	使用场景
DEBUG	10	开发调试信息（变量值、函数调用）
INFO	20	正常运行信息（服务启动、请求处理）
WARNING	30	潜在问题（配置缺失用默认值、连接重试）
ERROR	40	错误事件（请求失败、数据库异常）
CRITICAL	50	严重错误（服务崩溃、磁盘空间不足）

原则：开发环境用 `DEBUG`，生产环境用 `INFO`，只输出当前级别及以上的日志。

2.3 创建日志配置模块

文件：`backend/app/core/logger.py`

```
"""
统一日志配置模块

职责：
- 配置全局日志格式和输出级别
- 同时输出到控制台和日志文件
- 日志文件按大小自动轮转（保留最近 N 份）

使用方式：
在其他模块中引入：
    from app.core.logger import get_logger
    logger = get_logger(__name__)
    logger.info("服务启动成功")
"""

import logging
```

```

import os
import sys
from logging.handlers import RotatingFileHandler

from app.config.settings import settings

# — 日志目录 —————
# 日志文件存放在 backend/logs/ 目录下
LOG_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))),
                        "logs")
os.makedirs(LOG_DIR, exist_ok=True)

# — 日志格式 —————
# 格式说明:
#   %(asctime)s      — 时间戳（精确到毫秒）
#   %(levelname)-8s — 日志级别（左对齐，8 字符宽）
#   %(name)s        — Logger 名称（通常是模块名）
#   %(funcName)s    — 调用日志的函数名
#   %(lineno)d      — 调用日志的代码行号
#   %(message)s     — 日志内容
LOG_FORMAT = "%(asctime)s | %(levelname)-8s | %(name)s: %(funcName)s: %(lineno)d | %(message)s"
DATE_FORMAT = "%Y-%m-%d %H:%M:%S"

# — 格式化器 —————
formatter = logging.Formatter(LOG_FORMAT, datefmt=DATE_FORMAT)

# — 是否已初始化标记（避免重复添加 handler） —————
_initialized = False

def setup_logging():
    """
    初始化全局日志系统（只需调用一次）

    配置两个输出目标:
    1. 控制台 (StreamHandler) — 开发时实时查看
    2. 文件 (RotatingFileHandler) — 持久化存储，按大小轮转
    """
    global _initialized
    if _initialized:
        return
    _initialized = True

    # 获取根 Logger
    root_logger = logging.getLogger()
    root_logger.setLevel(getattr(logging, settings.LOG_LEVEL, logging.INFO))

# — 控制台 Handler —————
console_handler = logging.StreamHandler(sys.stdout)
console_handler.setLevel(logging.DEBUG)
console_handler.setFormatter(formatter)

```

```

root_logger.addHandler(console_handler)

# — 文件 Handler (轮转) —————
# maxBytes=10MB: 单个日志文件超过 10MB 时自动切割
# backupCount=5: 保留最近 5 份历史日志
# encoding="utf-8": 确保中文日志正常显示
file_path = os.path.join(LOG_DIR, "app.log")
file_handler = RotatingFileHandler(
    filename=file_path,
    maxBytes=10 * 1024 * 1024, # 10MB
    backupCount=5,
    encoding="utf-8",
)
file_handler.setLevel(logging.INFO) # 文件只记录 INFO 及以上
file_handler.setFormatter(formatter)
root_logger.addHandler(file_handler)

# — 降低第三方库日志级别, 避免刷屏 —————
logging.getLogger("uvicorn").setLevel(logging.WARNING)
logging.getLogger("sqlalchemy").setLevel(logging.WARNING)
logging.getLogger("minio").setLevel(logging.WARNING)
logging.getLogger("httpx").setLevel(logging.WARNING)

def get_logger(name: str) -> logging.Logger:
    """
    获取指定名称的 Logger

    Args:
        name: Logger 名称, 通常传入 __name__ (模块路径)

    Returns:
        配置好的 Logger 实例

    使用示例:
        logger = get_logger(__name__)
        logger.info("用户登录成功: user_id=%d", user_id)
        logger.error("数据库连接失败: %s", error_msg)
    """
    setup_logging() # 确保日志系统已初始化
    return logging.getLogger(name)

```

关键知识点讲解:

1. **RotatingFileHandler**: 自动轮转日志文件。当日志文件超过 `maxBytes` (10MB) 时, 自动重命名为 `app.log.1`, 新日志写入 `app.log`。最多保留 `backupCount` (5) 份, 自动删除最旧的
2. **get_logger(__name__)**: 传入 `__name__` 是 Python 最佳实践。Logger 名称自动反映模块路径, 如 `app.api.auth`, 方便定位日志来源
3. 降低第三方库日志级别: uvicorn、sqlalchemy 等库的 DEBUG 日志非常多, 设为 WARNING 避免干扰

2.4 创建 logs 目录

```
cd backend
mkdir -p logs
touch logs/.gitkeep
```

说明：`.gitkeep` 是一个空文件，用于让 Git 追踪空目录。日志文件本身不应提交到 Git。

在 `.gitignore` 中添加日志忽略规则（如果还没有的话）：

```
# 日志文件
logs/
*.log
```

2.5 在 settings.py 中添加日志配置项

文件：`backend/app/config/settings.py`（更新已有文件）

```
# 在 Settings 类中添加日志相关配置（已有 LOG_LEVEL，再补充文件配置）

# — 日志配置 —————
LOG_LEVEL: str = "INFO"          # 已有，日志级别
LOG_DIR: str = "logs"            # 日志目录（相对于 backend/）
LOG_MAX_BYTES: int = 10 * 1024 * 1024 # 单文件最大 10MB
LOG_BACKUP_COUNT: int = 5         # 保留 5 份历史日志
```

注意：`LOG_LEVEL` 已存在于 `settings.py` 中，只需确认其存在。新增的是 `LOG_DIR`、`LOG_MAX_BYTES`、`LOG_BACKUP_COUNT`。

2.6 使用示例

在任意模块中使用日志：

```
from app.core.logger import get_logger

logger = get_logger(__name__)

# 不同级别的使用场景
logger.debug("调试信息：变量 x = %s", x)          # 只在开发时输出
logger.info("用户 %s 登录成功", username)          # 正常运行信息
logger.warning("Redis 连接超时，正在重试...")      # 潜在问题
logger.error("数据库查询失败：%s", str(e))          # 错误事件
logger.critical("磁盘空间不足！剩余 %d MB", free)  # 严重错误
```

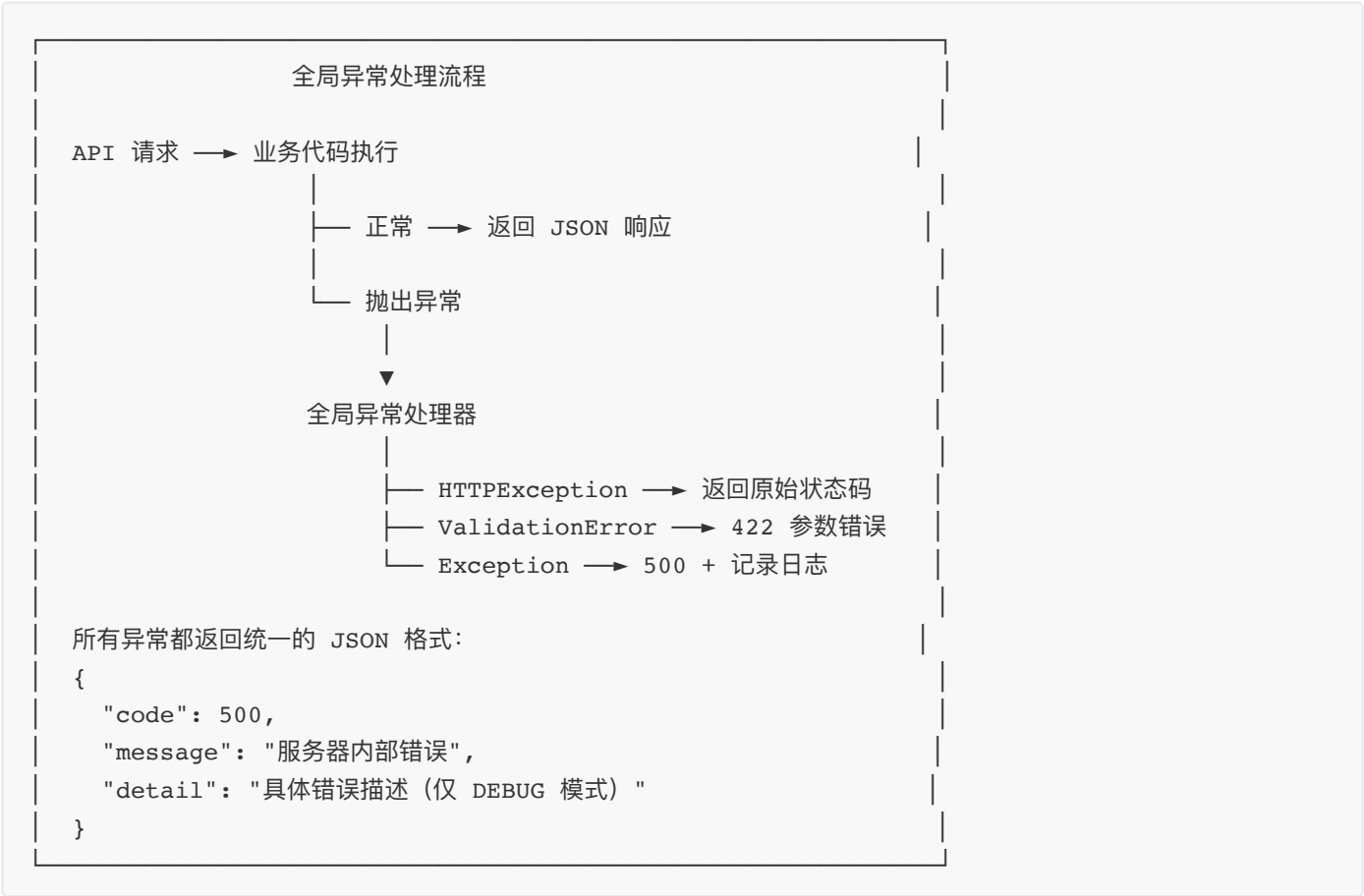
输出效果：

```
2026-06-30 09:30:15 | INFO      | app.api.auth:login:42 | 用户 testuser 登录成功
2026-06-30 09:30:16 | WARNING  | app.database.session:get_db:28 | Redis 连接超时，正在重试...
2026-06-30 09:30:17 | ERROR    | app.services.user_service:login:55 | 数据库查询失败：
connection refused
```

三、后端全局异常处理中间件

3.1 为什么需要全局异常处理

没有全局异常处理时，未捕获的异常会返回 FastAPI 默认的 `500 Internal Server Error`，前端只看到一个空泛的错误，无法定位问题。



3.2 创建全局异常处理模块

文件：`backend/app/core/exceptions.py`

```
"""
全局异常处理模块

职责：
- 捕获所有未处理的异常，返回统一 JSON 格式
- 区分已知异常（HTTP 业务错误）和未知异常（程序 Bug）
- 未知异常自动记录 ERROR 级别日志

注册方式（在 main.py 中）：
from app.core.exceptions import register_exception_handlers
register_exception_handlers(app)
"""

from fastapi import FastAPI, Request, HTTPException
```



```

from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from jose import JWTError

from app.core.logger import get_logger

logger = get_logger(__name__)

def register_exception_handlers(app: FastAPI):
    """注册全局异常处理器"""

    @app.exception_handler(HTTPException)
    async def http_exception_handler(request: Request, exc: HTTPException):
        """
        处理 HTTP 业务异常（如 400、401、404 等）

        这些是代码中主动 raise 的业务错误，不需要记录 ERROR 日志，
        只需返回给前端即可。
        """
        # 4xx 错误用 WARNING 级别（客户端问题）
        if 400 <= exc.status_code < 500:
            logger.warning(
                "HTTP %d: %s | path=%s",
                exc.status_code, exc.detail, request.url.path,
            )
        else:
            logger.error(
                "HTTP %d: %s | path=%s",
                exc.status_code, exc.detail, request.url.path,
            )

        return JSONResponse(
            status_code=exc.status_code,
            content={
                "code": exc.status_code,
                "message": exc.detail,
                "data": None,
            },
        )

    @app.exception_handler(RequestValidationError)
    async def validation_exception_handler(request: Request, exc: RequestValidationError):
        """
        处理 Pydantic 参数验证异常

        当请求参数不符合 Pydantic 模型定义时触发，
        如：缺少必填字段、类型不匹配、超出范围等。
        """
        # 提取所有验证错误信息
        errors = []
        for error in exc.errors():

```

```

        field = ".".join(str(loc) for loc in error["loc"] if loc != "body")
        errors.append(f"{field}: {error['msg']}")

    logger.warning(
        "参数验证失败 | path=%s | errors=%s",
        request.url.path, errors,
    )

    return JSONResponse(
        status_code=422,
        content={
            "code": 422,
            "message": "参数验证失败",
            "data": errors,
        },
    )

@app.exception_handler(JWTError)
async def jwt_exception_handler(request: Request, exc: JWTError):
    """处理 JWT Token 解析异常"""
    logger.warning("JWT 验证失败 | path=%s | error=%s", request.url.path, str(exc))

    return JSONResponse(
        status_code=401,
        content={
            "code": 401,
            "message": "无效的认证凭据",
            "data": None,
        },
    )

@app.exception_handler(Exception)
async def general_exception_handler(request: Request, exc: Exception):
    """
    处理所有未预期的异常（兜底处理器）

    这是最后一道防线：任何没有被上面三个处理器捕获的异常都会到这里。
    必须记录完整的错误堆栈，但返回给前端时隐藏细节（安全考虑）。
    """
    # 使用 exc_info=True 记录完整堆栈信息
    logger.error(
        "未处理的异常 | path=%s | method=%s | error=%s",
        request.url.path, request.method, str(exc),
        exc_info=True, # 关键：记录完整堆栈
    )

    return JSONResponse(
        status_code=500,
        content={
            "code": 500,
            "message": "服务器内部错误",
            "data": None,
        },
    )

```

```
    },  
    )
```

关键知识点讲解：

1. `@app.exception_handler(异常类型)`：FastAPI 装饰器，当代码中抛出指定类型的异常时，自动调用对应的处理函数
2. `exc_info=True`：在 `logger.error()` 中使用此参数，会自动记录完整的 Python 异常堆栈 (traceback)，便于排查问题
3. 安全原则：500 错误返回给前端时不暴露 `str(exc)` 详情，防止敏感信息泄露（如数据库连接字符串）
4. 统一响应格式：所有异常都返回 `{code, message, data}` 格式，与正常响应保持一致

3.3 在 main.py 中注册异常处理器

文件：`backend/main.py`（更新）

在创建 FastAPI 实例后，添加异常处理器注册：

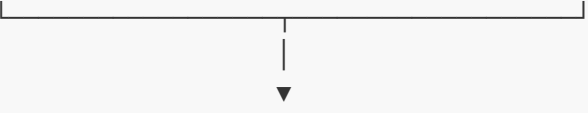
```
from app.core.exceptions import register_exception_handlers  
  
# ... 创建 app 后 ...  
  
# — 注册全局异常处理器 —————  
register_exception_handlers(app)
```

四、API 请求日志记录中间件

4.1 中间件工作原理

中间件（Middleware）是插入到请求处理流程中的一层"过滤器"，每个 API 请求都会经过它：





```
请求日志中间件（记录请求结束）
→ 记录：status_code, 耗时(ms)
```

4.2 创建中间件目录和模块

先创建目录：

```
cd backend/app
mkdir -p middleware
touch middleware/__init__.py
```

文件：`backend/app/middleware/request_logger.py`

```
"""
API 请求日志中间件

职责：
- 记录每个 API 请求的方法、路径、客户端 IP
- 记录请求耗时（毫秒）
- 记录响应状态码
- 自动跳过健康检查等高频接口（避免日志噪声）

注册方式（在 main.py 中）：
    from app.middleware.request_logger import RequestLogMiddleware
    app.add_middleware(RequestLogMiddleware)
"""

import time

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request

from app.core.logger import get_logger

logger = get_logger("request")

# 不记录日志的路径前缀（高频或无意义请求）
SKIP_PATHS = [
    "/docs",
    "/redoc",
    "/openapi.json",
    "/favicon.ico",
    "/api/health",  # 健康检查频率高，不记录
]
```

```

class RequestLogMiddleware(BaseHTTPMiddleware):
    """API 请求日志中间件"""

    async def dispatch(self, request: Request, call_next):
        """
        拦截每个请求，记录日志

        Args:
            request: 当前 HTTP 请求
            call_next: 调用链中的下一个处理器
        """
        # 检查是否需要跳过
        path = request.url.path
        if any(path.startswith(skip) for skip in SKIP_PATHS):
            return await call_next(request)

        # 记录请求开始
        method = request.method
        client_ip = request.client.host if request.client else "unknown"

        # 获取请求体大小 (如果有)
        content_length = request.headers.get("content-length", "0")

        logger.info(
            "→ %s %s | ip=%s | size=%s",
            method, path, client_ip, content_length,
        )

        # 记录开始时间
        start_time = time.time()

        # 调用下一个处理器 (执行业务逻辑)
        response = await call_next(request)

        # 计算耗时
        duration_ms = (time.time() - start_time) * 1000

        # 记录请求结束
        status_code = response.status_code

        # 根据状态码选择日志级别
        if status_code >= 500:
            logger.error(
                "← %s %s | status=%d | %.1fms",
                method, path, status_code, duration_ms,
            )
        elif status_code >= 400:
            logger.warning(
                "← %s %s | status=%d | %.1fms",
                method, path, status_code, duration_ms,
            )
        else:

```

```

        logger.info(
            "← %s %s | status=%d | %.1fms",
            method, path, status_code, duration_ms,
        )

    return response

```

关键知识点讲解：

1. `BaseHTTPMiddleware`：Starlette 提供的中间件基类，只需重写 `dispatch` 方法即可
2. `call_next(request)`：调用链中的下一个处理器。在它之前记录"请求开始"，在它之后记录"请求结束"
3. `time.time()`：通过前后时间差计算请求耗时
4. `SKIP_PATHS`：跳过 Swagger 文档和健康检查请求，避免日志被高频无用信息刷屏

4.3 在 main.py 中注册中间件

文件：`backend/main.py`（更新，在 CORS 中间件之后添加）

```

from app.middleware.request_logger import RequestLogMiddleware

# — 注册中间件（注意顺序） —————
# 中间件执行顺序：后添加的先执行（洋葱模型）
# 1. CORS 中间件（最先执行，处理跨域）
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 2. 请求日志中间件（在 CORS 之后执行）
app.add_middleware(RequestLogMiddleware)

```

4.4 日志输出效果

启动后端服务后，发起请求时控制台输出：

```

2026-06-30 10:15:32 | INFO      | request:dispatch:55 | → POST /api/auth/login |
ip=127.0.0.1 | size=52
2026-06-30 10:15:32 | INFO      | app.api.auth:login:68 | 用户 testuser 登录成功
2026-06-30 10:15:32 | INFO      | request:dispatch:73 | ← POST /api/auth/login | status=200
| 156.3ms
2026-06-30 10:15:35 | INFO      | request:dispatch:55 | → GET /api/auth/me | ip=127.0.0.1 |
size=0
2026-06-30 10:15:35 | INFO      | request:dispatch:73 | ← GET /api/auth/me | status=200 |
23.7ms

```

五、健康检查接口完善

5.1 为什么需要健康检查

健康检查接口是运维和监控系统的基础，用于：

- **Docker 容器编排**：判断服务是否存活（liveness probe）
- **负载均衡器**：判断后端实例是否可接收流量（readiness probe）
- **前端状态展示**：管理后台显示各服务运行状态

5.2 创建健康检查路由模块

将 main.py 中的健康检查逻辑拆分为独立模块：

文件：`backend/app/api/health.py`

```
"""
健康检查 API 路由

接口列表：
- GET /api/health          基础健康检查（应用状态）
- GET /api/health/detail   详细健康检查（含数据库、Redis、MinIO 状态）

设计原则：
- 基础检查不依赖外部服务，响应快
- 详细检查逐一验证各依赖服务的连通性
- 任一服务不可用时不抛异常，而是标记为 unhealthy
"""

from fastapi import APIRouter

from app.config.settings import settings
from app.core.logger import get_logger

logger = get_logger(__name__)

router = APIRouter(tags=["健康检查"])

@router.get("/api/health")
async def health_check():
    """
    基础健康检查

    用途：Docker liveness probe、负载均衡器探活
    特点：不检查外部依赖，只确认应用进程存活
    """
    return {
        "code": 200,
        "message": "ok",
        "data": {
```

```

        "status": "healthy",
        "app_name": settings.APP_NAME,
        "version": settings.APP_VERSION,
    },
}

```

```
@router.get("/api/health/detail")
```

```
async def health_check_detail():
```

```
    """
```

```
    详细健康检查
```

```
    用途：管理后台状态展示、运维监控
```

```
    特点：逐一检测 PostgreSQL、Redis、MinIO 连通性
```

```
    """
```

```
    services = {}
```

```
# — 检查 PostgreSQL —————
```

```
try:
```

```
    from sqlalchemy import text
```

```
    from app.database.session import SessionLocal
```

```
    db = SessionLocal()
```

```
    # 执行最简单的查询验证连接 (SQLAlchemy 2.x 语法)
```

```
    db.execute(text("SELECT 1"))
```

```
    db.close()
```

```
    services["database"] = {"status": "healthy", "message": "PostgreSQL 连接正常"}
```

```
except Exception as e:
```

```
    services["database"] = {"status": "unhealthy", "message": f"PostgreSQL 连接失败：
```

```
{str(e)}"}
```

```
    logger.error("PostgreSQL 健康检查失败：%s", str(e))
```

```
# — 检查 Redis —————
```

```
try:
```

```
    import redis
```

```
    r = redis.from_url(settings.REDIS_URL)
```

```
    r.ping()
```

```
    r.close()
```

```
    services["redis"] = {"status": "healthy", "message": "Redis 连接正常"}
```

```
except Exception as e:
```

```
    services["redis"] = {"status": "unhealthy", "message": f"Redis 连接失败：{str(e)}"}
```

```
    logger.error("Redis 健康检查失败：%s", str(e))
```

```
# — 检查 MinIO —————
```

```
try:
```

```
    from app.storage.minio_client import MinIOClient
```

```
    minio = MinIOClient()
```

```
    minio.client.list_buckets()
```

```
    services["minio"] = {"status": "healthy", "message": "MinIO 连接正常"}
```

```
except Exception as e:
```

```
    services["minio"] = {"status": "unhealthy", "message": f"MinIO 连接失败：{str(e)}"}
```

```
    logger.error("MinIO 健康检查失败：%s", str(e))
```



```
# — 汇总状态 —————
all_healthy = all(s["status"] == "healthy" for s in services.values())

return {
    "code": 200,
    "message": "ok",
    "data": {
        "status": "healthy" if all_healthy else "degraded",
        "app_name": settings.APP_NAME,
        "version": settings.APP_VERSION,
        "services": services,
    },
}
```

5.3 在 main.py 中注册健康检查路由

文件：backend/main.py (更新)

```
from app.api.health import router as health_router

# 注册路由
app.include_router(auth_router)
app.include_router(health_router)
```

同时删除 main.py 中原有的简单健康检查接口

(/api/health、/api/health/database、/api/health/redis、/api/health/minio)，因为已迁移到 health.py 中统一管理。

5.4 验证健康检查

```
# 基础健康检查
curl http://localhost:8000/api/health

# 预期输出：
# {"code":200,"message":"ok","data":{"status":"healthy","app_name":"RSOD Agent Platform","version":"0.1.0"}}

# 详细健康检查
curl http://localhost:8000/api/health/detail

# 预期输出（所有服务正常时）：
# {"code":200,"message":"ok","data":{"status":"healthy","app_name":"RSOD Agent Platform","version":"0.1.0","services":{"database":{"status":"healthy","message":"PostgreSQL 连接正常"},"redis":{"status":"healthy","message":"Redis 连接正常"},"minio":{"status":"healthy","message":"MinIO 连接正常"}}}}
```

六、前端错误监控与上报

6.1 前端错误类型

前端错误分类		
JavaScript 运行时错误	window.onerror 捕获	如：变量未定义、方法不存在
Promise 未捕获异常	window.onunhandledrejection 捕获	如：async 函数中 throw、API 调用失败
Vue 组件渲染错误	app.config.errorHandler 捕获	如：模板表达式错误、生命周期钩子异常
资源加载错误	通过事件监听捕获	如：图片 404、脚本加载失败

6.2 创建前端错误上报模块

文件：`frontend/src/utils/errorReporter.js`

```
/**
 * 前端全局错误监控与上报
 *
 * 职责：
 *   - 捕获 JavaScript 运行时错误
 *   - 捕获未处理的 Promise 异常
 *   - 捕获 Vue 组件渲染错误
 *   - 将错误信息上报到后端（用于分析和告警）
 *
 * 使用方式（在 main.js 中）：
 *   import { setupErrorReporting } from "@utils/errorReporter";
 *   setupErrorReporting(app);
 */

import { ElMessage } from "element-plus";

// — 错误上报地址 —————
// Day 4 先上报到控制台 + 本地存储，Day 11 接入后端 API
const REPORT_TO_BACKEND = false; // 生产环境设为 true
const REPORT_API = "/api/errors/report";

/**
 * 上报错误信息
 * 当前实现：输出到控制台 + 存入 localStorage
 * 生产环境可发送到后端日志收集服务
 */
function reportError(errorInfo) {
  // 1. 控制台输出（开发调试）
```

```

console.error("[ErrorReporter]", errorInfo);

// 2. 存入本地存储 (最近 50 条错误)
try {
  const errors = JSON.parse(localStorage.getItem("error_logs") || "[]");
  errors.push({
    ...errorInfo,
    timestamp: new Date().toISOString(),
    url: window.location.href,
    userAgent: navigator.userAgent,
  });
  // 只保留最近 50 条
  if (errors.length > 50) {
    errors.splice(0, errors.length - 50);
  }
  localStorage.setItem("error_logs", JSON.stringify(errors));
} catch (e) {
  // localStorage 写入失败不影响程序运行
  console.warn("ErrorReporter: localStorage 写入失败", e);
}

// 3. 上报到后端 (生产环境启用)
if (REPORT_TO_BACKEND) {
  fetch(REPORT_API, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(errorInfo),
  }).catch(() => {
    // 上报失败时静默处理, 不能因为上报失败而再次报错
  });
}
}

/**
 * 初始化全局错误监控
 * @param {import('vue').App} app - Vue 应用实例
 */
export function setupErrorReporting(app) {
  // — 1. Vue 组件错误 —————
  app.config.errorHandler = (err, instance, info) => {
    reportError({
      type: "vue_error",
      message: err.message,
      stack: err.stack,
      component: info, // 错误发生所在组件的生命周期钩子
    });

    // 给用户友好提示
    ElMessage.error("页面渲染出错, 请刷新重试");
  };

  // — 2. JavaScript 运行时错误 —————

```

```

window.onerror = (message, source, lineno, colno, error) => {
  reportError({
    type: "js_error",
    message: message,
    source: source,
    lineno: lineno,
    colno: colno,
    stack: error?.stack,
  });
};

// — 3. 未捕获的 Promise 异常 —————
window.onunhandledrejection = (event) => {
  reportError({
    type: "promise_rejection",
    message: event.reason?.message || String(event.reason),
    stack: event.reason?.stack,
  });

  // 阻止默认的浏览器控制台输出（已自行处理）
  event.preventDefault();
};

console.log("[ErrorReporter] 全局错误监控已启用");
}

```

6.3 在 main.js 中注册

文件：frontend/src/main.js（更新）

```

import { setupErrorReporting } from "@utils/errorReporter";

// — 创建并配置应用 —————
const app = createApp(App);

// 注册全局错误监控（在其他插件之前注册）
setupErrorReporting(app);

// 注册插件
app.use(pinia);
app.use(router);
app.use(ElementPlus, { locale: zhCn });

// 挂载到 DOM
app.mount("#app");

```

6.4 查看本地错误日志

在浏览器控制台中执行以下代码，可查看本地存储的错误日志：

```
JSON.parse(localStorage.getItem("error_logs"))
```

七、Docker 容器日志查看与管理

7.1 Docker 日志基础命令

```
# — 查看容器实时日志 —————
# 查看所有服务的日志（实时滚动）
docker compose logs -f

# 只查看某个服务的日志
docker compose logs -f postgres
docker compose logs -f redis
docker compose logs -f minio

# — 查看最近的 N 行日志 —————
docker compose logs --tail=50 postgres

# — 查看指定时间段内的日志 —————
docker compose logs --since="2026-06-30T09:00:00" postgres

# — 带时间戳查看 —————
docker compose logs -f -t redis
```

7.2 各服务日志说明

Docker 容器日志一览	
postgres	数据库启动信息、连接记录、慢查询日志 关注：FATAL（启动失败）、ERROR（连接/查询错误）
redis	启动信息、内存使用、持久化状态 关注：WARNING（内存不足）、OOM（内存溢出）
minio	API 请求日志、存储操作日志 关注：ERROR（上传/下载失败）
后端应用	不通过 Docker 运行，直接 python main.py 日志输出到终端 + logs/app.log 文件

7.3 常用排查场景

场景 1：数据库连接失败

```
# 查看 PostgreSQL 是否正常运行
docker compose ps postgres
# 如果状态不是 healthy, 查看详细日志
docker compose logs postgres
```

常见错误:

- `FATAL: password authentication failed` → 检查 `.env` 中的密码配置
- `database "rsod_agent" does not exist` → 检查 `docker-compose.yml` 中的 `POSTGRES_DB`

场景 2: Redis 连接超时

```
# 检查 Redis 状态
docker compose logs --tail=20 redis
# 尝试手动连接
docker compose exec redis redis-cli ping
```

场景 3: MinIO 存储异常

```
# 查看 MinIO 日志
docker compose logs minio
# 检查存储桶是否存在
docker compose exec minio mc ls local/rsod-agent-images 2>/dev/null || echo "存储桶检查需要 mc 客户端"
```

7.4 日志文件管理

后端应用日志文件位置:

```
# 查看日志文件
ls -lh backend/logs/

# 查看最新日志
tail -f backend/logs/app.log

# 查看错误日志 (过滤 ERROR 级别)
grep "ERROR" backend/logs/app.log

# 统计各级别日志数量
grep -c "INFO" backend/logs/app.log
grep -c "WARNING" backend/logs/app.log
grep -c "ERROR" backend/logs/app.log
```

八、pytest 测试框架搭建

8.1 为什么需要自动化测试

手动测试 vs 自动化测试	
手动测试	打开 Swagger → 输入参数 → 点击执行 → 看结果 问题：重复劳动、容易遗漏、无法持续执行
自动化测试	编写测试代码 → pytest 一键运行 → 自动验证 优势：可重复、可集成到 CI/CD、覆盖率高

8.2 安装测试依赖

```
cd backend
source .venv/bin/activate

# 安装测试相关包
pip install pytest pytest-asyncio httpx
```

更新 `backend/requirements.txt`，在末尾追加：

```
# — 测试 —————
pytest==8.0.0
pytest-asyncio==0.23.0
httpx==0.27.0
```

说明：`httpx` 已在依赖列表中（用于 HTTP 请求），这里复用它作为 FastAPI 的测试客户端。`pytest-asyncio` 用于测试异步函数。

8.3 创建测试目录结构

```
cd backend
mkdir -p tests
touch tests/__init__.py
```

目标结构：

```
backend/
├── tests/
│   ├── __init__.py      # 标记为 Python 包
│   ├── conftest.py      # pytest 全局 fixtures（测试客户端、数据库等）
│   ├── test_health.py   # 健康检查接口测试
│   └── test_auth.py     # 认证接口测试
```

8.4 创建 pytest 配置文件

文件：`backend/pytest.ini`

```
# pytest 配置文件
# 文档: https://docs.pytest.org/en/stable/reference/customize.html

[pytest]
# 测试文件搜索模式
testpaths = tests

# 异步测试模式
asyncio_mode = auto

# 输出格式: -v 显示每个测试名, -s 允许 print 输出
addopts = -v --tb=short

# 过滤警告 (减少无关信息)
filterwarnings =
    ignore::DeprecationWarning
    ignore::UserWarning
```

8.5 创建 conftest.py (全局 Fixtures)

文件: `backend/tests/conftest.py`

```
"""
pytest 全局 Fixtures

Fixtures 是 pytest 的核心概念, 用于:
- 创建测试所需的前置条件 (如数据库连接、测试客户端)
- 在所有测试用例间共享资源
- 自动清理测试产生的数据

conftest.py 中的 fixtures 对所有测试文件可用, 无需显式导入。
"""

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.database.session import Base, get_db

# — 测试数据库 —————
# 使用 SQLite 内存数据库进行测试, 避免依赖 PostgreSQL
# 优点: 速度快、隔离性好、无需清理
# 注意: SQLite 不支持 PostgreSQL 特有功能 (如 JSON 字段),
# 但对于基础 CRUD 测试足够
TEST_DATABASE_URL = "sqlite:///./test.db"

test_engine = create_engine(
    TEST_DATABASE_URL,
    connect_args={"check_same_thread": False}, # SQLite 特有参数
)
```



```

TestSessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=test_engine,
)

def override_get_db():
    """覆盖 FastAPI 的 get_db 依赖，使用测试数据库"""
    db = TestSessionLocal()
    try:
        yield db
    finally:
        db.close()

# — 导入所有模型（确保 Base.metadata 包含所有表定义） —
from app.entity import db_models # noqa: E402, F401
from main import app # noqa: E402

# 覆盖依赖
app.dependency_overrides[get_db] = override_get_db

@pytest.fixture(scope="session", autouse=True)
def setup_test_database():
    """
    创建测试数据库表（所有测试共享）

    scope="session": 整个测试会话只执行一次
    autouse=True: 自动应用，无需在测试函数中显式引用
    """
    Base.metadata.create_all(bind=test_engine)
    yield
    # 测试结束后清理
    Base.metadata.drop_all(bind=test_engine)
    # 删除 SQLite 文件
    import os
    if os.path.exists("./test.db"):
        os.remove("./test.db")

@pytest.fixture
def client():
    """
    提供 FastAPI 测试客户端

    用法:
    def test_health(client):
        response = client.get("/api/health")
        assert response.status_code == 200
    """

```

```

"""
return TestClient(app)

@pytest.fixture
def db_session():
    """
    提供独立的数据库会话（每个测试用独立事务）

    用法：
    def test_create_user(db_session):
        user = User(username="test", ...)
        db_session.add(user)
        db_session.commit()
    """
    session = TestSessionLocal()
    try:
        yield session
    finally:
        session.rollback()
        session.close()

```

关键知识点讲解：

1. **TestClient**：FastAPI 提供的测试客户端（底层是 httpx），可以直接调用 API 而无需启动服务器
2. **dependency_overrides**：FastAPI 的依赖覆盖机制，将 `get_db` 替换为使用测试数据库的版本
3. **SQLite 内存数据库**：测试使用独立的 SQLite 数据库，不影响开发环境的 PostgreSQL
4. **@pytest.fixture**：定义可复用的测试前置条件，`scope="session"` 表示整个测试过程只创建一次

8.6 编写健康检查测试

文件：`backend/tests/test_health.py`

```

"""
健康检查接口测试

测试目标：
- GET /api/health 返回正确的状态和格式
- GET / 返回欢迎信息
"""

def test_health_check(client):
    """测试基础健康检查接口"""
    response = client.get("/api/health")

    # 验证状态码
    assert response.status_code == 200

    # 验证响应格式

```

```

data = response.json()
assert data["code"] == 200
assert data["message"] == "ok"
assert data["data"]["status"] == "healthy"
assert data["data"]["app_name"] == "RSOD Agent Platform"
assert "version" in data["data"]

def test_root(client):
    """测试根路径欢迎接口"""
    response = client.get("/")

    assert response.status_code == 200
    data = response.json()
    assert "message" in data
    assert "version" in data
    assert "docs" in data

```

8.7 编写认证接口测试

文件：backend/tests/test_auth.py

```

"""
认证接口测试

测试目标：
- 用户注册：正常注册、重复用户名、参数验证
- 用户登录：正常登录、错误密码、不存在的用户
- 获取当前用户：有 Token、无 Token、无效 Token

测试策略：
- 每个测试用例独立，不依赖其他测试的执行顺序
- 使用唯一的用户名避免测试间冲突
"""

import pytest

class TestRegister:
    """用户注册测试"""

    def test_register_success(self, client):
        """正常注册"""
        response = client.post(
            "/api/auth/register",
            json={
                "username": "test_register_user",
                "email": "test_register@example.com",
                "password": "123456",
            },
        )

```

```

assert response.status_code == 201
data = response.json()
assert data["username"] == "test_register_user"
assert data["email"] == "test_register@example.com"
# 确保不返回密码字段
assert "hashed_password" not in data
assert "password" not in data

def test_register_duplicate_username(self, client):
    """重复用户名注册"""
    # 先注册一个用户
    client.post(
        "/api/auth/register",
        json={
            "username": "dup_user",
            "email": "dup1@example.com",
            "password": "123456",
        },
    )
    # 用相同用户名再注册
    response = client.post(
        "/api/auth/register",
        json={
            "username": "dup_user",
            "email": "dup2@example.com",
            "password": "123456",
        },
    )
    assert response.status_code == 400

def test_register_short_username(self, client):
    """用户名过短 (少于 3 字符) """
    response = client.post(
        "/api/auth/register",
        json={
            "username": "ab",
            "email": "short@example.com",
            "password": "123456",
        },
    )
    assert response.status_code == 422

def test_register_short_password(self, client):
    """密码过短 (少于 6 位) """
    response = client.post(
        "/api/auth/register",
        json={
            "username": "short_pwd_user",
            "email": "shortpwd@example.com",
            "password": "123",
        },
    )

```

```

        assert response.status_code == 422

def test_register_missing_fields(self, client):
    """缺少必填字段"""
    response = client.post(
        "/api/auth/register",
        json={"username": "no_email_user"},
    )
    assert response.status_code == 422

class TestLogin:
    """用户登录测试"""

def test_login_success(self, client):
    """正常登录"""
    # 先注册
    client.post(
        "/api/auth/register",
        json={
            "username": "login_user",
            "email": "login@example.com",
            "password": "123456",
        },
    )
    # 再登录
    response = client.post(
        "/api/auth/login",
        json={
            "username": "login_user",
            "password": "123456",
        },
    )
    assert response.status_code == 200
    data = response.json()
    assert "access_token" in data
    assert data["token_type"] == "bearer"
    assert data["user"]["username"] == "login_user"

def test_login_wrong_password(self, client):
    """密码错误"""
    # 先注册
    client.post(
        "/api/auth/register",
        json={
            "username": "wrong_pwd_user",
            "email": "wrongpwd@example.com",
            "password": "123456",
        },
    )
    # 用错误密码登录
    response = client.post(

```

```

        "/api/auth/login",
        json={
            "username": "wrong_pwd_user",
            "password": "wrongpassword",
        },
    )
    assert response.status_code == 401

def test_login_nonexistent_user(self, client):
    """不存在的用户"""
    response = client.post(
        "/api/auth/login",
        json={
            "username": "no_such_user_12345",
            "password": "123456",
        },
    )
    assert response.status_code == 401

class TestGetCurrentUser:
    """获取当前用户测试"""

    def test_get_me_with_valid_token(self, client):
        """使用有效 Token 获取用户信息"""
        # 注册并登录
        client.post(
            "/api/auth/register",
            json={
                "username": "me_user",
                "email": "me@example.com",
                "password": "123456",
            },
        )
        login_response = client.post(
            "/api/auth/login",
            json={
                "username": "me_user",
                "password": "123456",
            },
        )
        token = login_response.json()["access_token"]

        # 使用 Token 获取用户信息
        response = client.get(
            "/api/auth/me",
            headers={"Authorization": f"Bearer {token}"},
        )
        assert response.status_code == 200
        data = response.json()
        assert data["username"] == "me_user"
        assert data["email"] == "me@example.com"

```

```

def test_get_me_without_token(self, client):
    """不带 Token 访问受保护接口"""
    response = client.get("/api/auth/me")
    assert response.status_code == 401

def test_get_me_with_invalid_token(self, client):
    """使用无效 Token"""
    response = client.get(
        "/api/auth/me",
        headers={"Authorization": "Bearer invalid_token_here"},
    )
    assert response.status_code == 401

```

8.8 运行测试

```

cd backend
source .venv/bin/activate

# 运行所有测试
pytest

# 运行指定测试文件
pytest tests/test_health.py

# 运行指定测试类
pytest tests/test_auth.py::TestRegister

# 运行指定测试方法
pytest tests/test_auth.py::TestLogin::test_login_success

# 详细输出模式
pytest -v -s

# 显示每个测试的耗时
pytest --durations=10

```

预期输出：

```

tests/test_health.py::test_health_check PASSED [ 12%]
tests/test_health.py::test_root PASSED [ 25%]
tests/test_auth.py::TestRegister::test_register_success PASSED [ 37%]
tests/test_auth.py::TestRegister::test_register_duplicate_username PASSED [ 50%]
tests/test_auth.py::TestRegister::test_register_short_username PASSED [ 62%]
tests/test_auth.py::TestRegister::test_register_short_password PASSED [ 75%]
tests/test_auth.py::TestRegister::test_register_missing_fields PASSED [ 87%]
tests/test_auth.py::TestLogin::test_login_success PASSED [ 93%]
...

===== 13 passed in 2.35s =====

```

九、API 接口测试编写

(已包含在上面的 `test_auth.py` 中。以下补充一些测试最佳实践。)

9.1 测试组织原则

测试组织规范	
文件命名	<code>test_<模块名>.py</code> 如: <code>test_auth.py</code> , <code>test_detection.py</code>
类命名	<code>Test<功能名></code> 如: <code>TestRegister</code> , <code>TestLogin</code>
方法命名	<code>test_<场景描述></code> 如: <code>test_login_success</code> , <code>test_login_wrong</code>
测试结构	AAA 模式 Arrange (准备) → Act (执行) → Assert (断言)

9.2 常用断言方法

```
# — 状态码断言 —————
assert response.status_code == 200      # 成功
assert response.status_code == 201      # 创建成功
assert response.status_code == 400      # 请求参数错误
assert response.status_code == 401      # 未认证
assert response.status_code == 404      # 资源不存在
assert response.status_code == 422      # 参数验证失败

# — 响应内容断言 —————
data = response.json()
assert data["code"] == 200              # 业务状态码
assert "access_token" in data           # 包含某字段
assert "password" not in data           # 不包含敏感字段
assert len(data["data"]) > 0            # 列表非空
assert isinstance(data["data"], list)    # 类型检查

# — 数据库断言 —————
from app.entity.db_models import User
user = db_session.query(User).filter(User.username == "test").first()
assert user is not None                 # 记录存在
assert user.email == "test@example.com" # 字段值正确
```

十、前端单元测试基础 (Vitest)

10.1 什么是 Vitest

Vitest 是 Vite 生态的测试框架，与 Jest 语法兼容，但基于 Vite 构建，速度更快。

Vitest vs Jest 对比	
构建工具	Vitest 基于 Vite (ESM 原生) Jest 基于 Webpack/Babel
配置	复用 vite.config.js, 零额外配置 需要独立的 jest.config.js
速度	利用 Vite HMR, 增量测试极快 每次全量编译
语法	与 Jest 完全兼容 (describe/it/expect)

Vitest 测试的核心意义：用自动化代码证明你的代码是正确的，并防止未来的修改破坏已有的功能。

为什么需要测试

想象一个场景：

开发阶段：
你写了一个登录功能 → 手动测试了几次 → 感觉没问题 → 提交代码

一周后：
同事修改了 Token 存储逻辑 → 没意识到影响登录 → 登录功能悄悄坏了

上线后：
用户反馈登录失败 → 排查问题花了 2 小时 → 用户流失

如果有了测试：

开发阶段：
你写了登录功能 + 登录测试 → 测试通过 → 提交代码

一周后：
同事修改 Token 存储 → 运行测试 → 测试失败 → 立即发现问题 → 修复

上线后：
所有测试通过 → 放心发布 → 零事故

测试的五大价值

测试的五大价值		
1. 防止回归缺陷	修改代码后，测试自动验证没有破坏旧功能	
2. 快速定位问题	测试失败时，精确告诉你哪行代码出了问题	
3. 提升代码质量	写测试时会发现代码设计缺陷，迫使你重构	
4. 文档作用	测试用例本身就是活文档，展示代码的使用方式	
5. 加速开发	不用反复手动测试，可以快速尝试各种方案	

覆盖率的意义

覆盖率就像一张地图，告诉你：

- ✓ 哪些代码被测试覆盖了（绿色区域）
- x 哪些代码没有被测试覆盖（红色区域）

但要注意：

- 高覆盖率 ≠ 高质量测试
- 覆盖率只是一个参考指标，不是目标本身

真正重要的是：

- 测试是否覆盖了关键业务场景？
- 测试是否覆盖了边界条件？
- 测试是否覆盖了错误路径？

10.2 安装 Vitest

```
cd frontend

# 安装 Vitest 和 Vue 测试工具
npm install -D vitest @vue/test-utils happy-dom
```

- `vitest`：测试框架
- `@vue/test-utils`：Vue 组件测试工具
- `happy-dom`：轻量级 DOM 模拟（比 jsdom 更快）

10.3 配置 Vitest

文件：`frontend/vite.config.js`（更新）

```
import vue from "@vitejs/plugin-vue";
import { defineConfig } from "vite";
import path from "path";
import { fileURLToPath } from "url";
```

```

const __dirname = path.dirname(fileURLToPath(import.meta.url));

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },

  // — CSS 预处理器配置 —————
  css: {
    preprocessorOptions: {
      scss: {
        additionalData: `@use "@assets/styles/variables.scss" as *;`,
      },
    },
  },

  // — 开发服务器配置 —————
  server: {
    port: 5173,
    open: true,
    proxy: {
      "/api": {
        target: "http://localhost:8000",
        changeOrigin: true,
      },
    },
  },

  // — Vitest 测试配置 —————
  test: {
    // 使用 happy-dom 模拟浏览器环境
    environment: "happy-dom",
    // 全局 setup 文件
    setupFiles: ["../tests/setup.js"],
    // 测试文件匹配模式
    include: ["tests/**/*.{test,spec}.{js,ts}"],
    // 覆盖率 (可选)
    coverage: {
      provider: "v8",
      reporter: ["text", "html"],
    },
  },
});

```

10.4 创建测试 setup 文件

文件: `frontend/tests/setup.js`

```

/**
 * Vitest 全局 setup
 * 在每个测试文件执行前自动运行
 */

// 模拟 localStorage (happy-dom 已内置)
// 如需额外 mock, 在此添加

// 模拟 Element Plus 的 ElMessage (避免测试中弹出消息框)
import { vi } from "vitest";

vi.mock("element-plus", async () => {
  const actual = await vi.importActual("element-plus");
  return {
    ...actual,
    ElMessage: {
      success: vi.fn(),
      error: vi.fn(),
      warning: vi.fn(),
      info: vi.fn(),
    },
  };
});

```

10.5 在 package.json 中添加测试脚本

文件: `frontend/package.json` (更新)

```

{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "vitest",
    "test:run": "vitest run",
    "test:coverage": "vitest run --coverage"
  }
}

```

10.6 编写工具函数测试

文件: `frontend/tests/utils/request.test.js`

```

/**
 * Axios 请求封装测试
 *
 * 测试目标:
 *   - Axios 实例创建正确
 *   - 请求拦截器正常注入 Token
 *   - 响应拦截器正确处理错误

```

```

*/

import { describe, it, expect, vi, beforeEach } from "vitest";

describe("Axios 请求封装", () => {
  beforeEach(() => {
    // 每个测试前清除 mock
    vi.clearAllMocks();
    // 清除 localStorage
    localStorage.clear();
  });

  it("应该正确创建 axios 实例", async () => {
    const { default: request } = await import("@/utils/request");
    expect(request).toBeDefined();
    expect(request.defaults.baseURL).toBe("/api");
    expect(request.defaults.timeout).toBe(30000);
  });

  it("请求拦截器应该设置 Content-Type", async () => {
    const { default: request } = await import("@/utils/request");
    expect(request.defaults.headers["Content-Type"]).toBe("application/json");
  });
});

describe("错误上报模块", () => {
  it("应该正确初始化错误上报", async () => {
    const { setupErrorReporting } = await import("@/utils/errorReporter");
    expect(setupErrorReporting).toBeDefined();
    expect(typeof setupErrorReporting).toBe("function");
  });

  it("错误信息应该存入 localStorage", () => {
    // 模拟错误上报
    const errorInfo = {
      type: "test_error",
      message: "测试错误",
    };

    // 手动触发上报逻辑
    const errors = JSON.parse(localStorage.getItem("error_logs") || "[]");
    errors.push({ ...errorInfo, timestamp: new Date().toISOString() });
    localStorage.setItem("error_logs", JSON.stringify(errors));

    // 验证
    const stored = JSON.parse(localStorage.getItem("error_logs"));
    expect(stored).toHaveLength(1);
    expect(stored[0].type).toBe("test_error");
  });
});

```

10.7 编写 Vue 组件测试（示例）

文件：`frontend/tests/components/AppHeader.test.js`

```
/**
 * AppHeader 组件测试（示例）
 *
 * 注意：组件测试需要完整模拟 Element Plus 和 Router，
 * Day 4 只展示基础写法，后续 Days 会完善更多组件测试。
 */

import { describe, it, expect, vi } from "vitest";

// Day 4 先测试工具函数，组件测试在后续 Day 中完善
describe("AppHeader 组件", () => {
  it("组件文件应该存在", async () => {
    // 验证组件文件可被导入
    const module = await import("@/components/layout/AppHeader.vue");
    expect(module.default).toBeDefined();
  });
});
```

先创建目录：

```
cd frontend
mkdir -p tests/components
```

10.8 运行前端测试

```
cd frontend

# 运行所有测试（watch 模式，修改后自动重跑）
npm test

# 运行一次（CI/CD 模式）
npm run test:run

# 生成覆盖率报告（需要先安装install @vitest/coverage-v8）
npm install @vitest/coverage-v8
#生成覆盖率报告
npm run test:coverage
```

预期输出：

```
✓ tests/utils/request.test.js (2 tests) 23ms
  ✓ Axios 请求封装 > 应该正确创建 axios 实例
  ✓ Axios 请求封装 > 请求拦截器应该设置 Content-Type

✓ tests/utils/request.test.js (2 tests) 15ms
```

```
✓ 错误上报模块 > 应该正确初始化错误上报
✓ 错误上报模块 > 错误信息应该存入 localStorage

✓ tests/components/AppHeader.test.js (1 test) 8ms
  ✓ AppHeader 组件 > 组件文件应该存在

Test Files  2 passed (2)
  Tests     5 passed (5)
Duration    523ms
```

覆盖率报告字段说明：

列名	说明
File	被测文件路径
% Stmts	语句覆盖率：有多少百分比的代码语句被执行过
% Branch	分支覆盖率：有多少百分比的 if/else 分支被执行过
% Funcs	函数覆盖率：有多少百分比的函数被调用过
% Lines	行覆盖率：有多少百分比的代码行被执行过
Uncovered Line #s	未覆盖的行号：哪些代码行没有被测试覆盖

十一、前后端完整联调验证

11.1 联调检查清单

Day 4 是第一阶段收尾，需要确保前后端完整打通：

前后端联调检查清单		
1. 后端服务启动	→	http://localhost:8000/docs 可访问
2. 前端服务启动	→	http://localhost:5173 可访问
3. 注册功能	→	前端注册页面提交成功，获取用户信息
4. 登录功能	→	前端登录成功，获取 Token，跳转主界面
5. Token 自动注入	→	后续请求自动携带 Authorization Header
6. 路由守卫	→	未登录访问受保护页面自动跳转登录页
7. 侧边栏导航	→	点击各页面路由可正常切换
8. 各页面可访问	→	对话/检测/训练/历史页面渲染正常
9. 错误处理	→	后端异常返回友好 JSON，前端展示错误提示
10. 日志输出	→	后端控制台有请求日志，文件有日志记录

11.2 启动服务

```
# 终端 1: 启动基础设施
cd rsod-agent-platform
docker compose up -d postgres redis minio

# 终端 2: 启动后端
cd backend
source .venv/bin/activate
python main.py

# 终端 3: 启动前端
cd frontend
npm run dev
```

11.3 逐步联调验证

步骤 1: 验证后端

```
# 健康检查
curl http://localhost:8000/api/health
# 预期: {"code":200,"message":"ok","data":{"status":"healthy",...}}
```

步骤 2: 验证前端

浏览器访问 `http://localhost:5173`, 应看到:

- 自动跳转到登录页 (因为未登录)
- 登录页和注册页渲染正常

步骤 3: 注册 → 登录 → 主界面

1. 在注册页面填写信息并提交
2. 自动跳转到登录页
3. 使用注册的账号登录
4. 成功跳转到主界面 (Dashboard 或 Chat 页面)
5. 侧边栏显示所有导航项

步骤 4: 验证页面导航

依次点击侧边栏:

- 智能对话 — 页面正常渲染
- 目标检测 — 页面正常渲染
- 模型训练 — 页面正常渲染
- 历史记录 — 页面正常渲染

步骤 5: 验证 Token 机制

1. 打开浏览器开发者工具 (F12) → Network 面板

- 2. 点击任意侧边栏导航
- 3. 查看请求头中是否包含 `Authorization: Bearer eyJhbG...`

步骤 6：验证错误处理

- 1. 尝试使用错误的密码登录
- 2. 前端应显示"用户名或密码错误"的提示消息
- 3. 后端控制台应输出 WARNING 级别日志

11.4 联调验证截图清单

以下为 Day 4 验收需要截图保存的内容：

序号	截图内容	说明
1	后端 Swagger 文档页	证明后端服务正常运行
2	前端登录页面	证明前端服务正常运行
3	登录成功后的主界面	证明前后端联调通过
4	侧边栏导航切换	证明路由系统正常
5	后端控制台请求日志	证明日志系统工作正常
6	logs/app.log 文件内容	证明日志文件写入正常
7	pytest 测试结果	证明后端测试框架可用
8	Vitest 测试结果	证明前端测试框架可用

十二、常见问题排查

Q1：日志文件未生成

原因：`logs/` 目录不存在或权限不足。

解决：

```
cd backend
mkdir -p logs
chmod 755 logs
```

Q2：pytest 运行报错 `ModuleNotFoundError: No module named 'app'`

原因：在错误目录执行 pytest，或虚拟环境未激活。

解决：

```
cd backend          # 确保在 backend 目录下
source .venv/bin/activate
pytest
```

Q3: pytest 报错 `sqlite3.OperationalError: no such table: users`

原因：测试数据库未正确初始化。

排查：检查 `conftest.py` 中是否正确导入了 `db_models`：

```
# conftest.py 中必须有这行导入
from app.entity import db_models # noqa: E402, F401
```

Q4: 中间件注册后请求报错 `TypeError: object is not callable`

原因：中间件类没有正确继承 `BaseHTTPMiddleware`。

解决：确认 `RequestLogMiddleware` 继承自 `starlette.middleware.base.BaseHTTPMiddleware`。

Q5: 前端 Vitest 运行报错 `Cannot find module`

原因：测试文件中的 `@/` 路径别名未正确解析。

解决：确认 `vite.config.js` 中的 `resolve.alias` 配置正确，且 `test` 配置中的 `include` 模式正确。

Q6: 健康检查详细接口中 PostgreSQL 报错

原因：SQLAlchemy 2.x 的 `execute` 方法签名变更。

解决：修改 `health.py` 中的 PostgreSQL 检查逻辑：

```
# 替代方案：使用 text() 执行原生 SQL
from sqlalchemy import text
db = SessionLocal()
db.execute(text("SELECT 1"))
db.close()
```

Q7: 前端 EMessage 未定义报错

原因：`errorReporter.js` 中引入 `EMessage` 但 `Element Plus` 尚未加载。

解决：确保 `setupErrorReporting(app)` 在 `app.use(ElementPlus)` 之后调用，或将 `errorHandler` 中的 `EMessage.error` 改为可选调用：

```
try {
  ElMessage?.error("页面渲染出错，请刷新重试");
} catch {
  // Element Plus 未加载时静默处理
}
```

Q8: pytest 测试间数据冲突（重复用户名）

原因：SQLite 测试数据库在整个测试会话期间共享，前一个测试创建的用户在后续测试中仍存在。

解决：每个测试用例使用唯一的用户名（如加随机后缀），或在 `conftest.py` 中使用 `function` 作用域的 fixture 来重置数据库：

```
@pytest.fixture(autouse=True)
def clean_database(db_session):
    """每个测试后清理数据"""
    yield
    # 回滚所有未提交的更改
    db_session.rollback()
    # 清理所有表数据
    for table in reversed(Base.metadata.sorted_tables):
        db_session.execute(table.delete())
    db_session.commit()
```

Day 4 验收自查清单

逐项确认以下全部通过：

```
# == 后端验证 ==

# 1. 日志系统工作正常
cd backend
source .venv/bin/activate
python main.py
# 在另一个终端发起请求，观察控制台是否输出格式化日志
# 检查 logs/app.log 文件是否有日志记录

# 2. 全局异常处理正常
curl -X POST http://localhost:8000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"ab","email":"test@test.com","password":"123456"}'
# 预期：返回 422 + 友好 JSON 错误（不是原始 500 错误）

# 3. 健康检查接口
curl http://localhost:8000/api/health
curl http://localhost:8000/api/health/detail

# 4. pytest 测试通过
cd backend
```

```
pytest
# 预期: 所有测试 PASSED

# == 前端验证 ==

# 5. Vitest 测试通过
cd frontend
npm run test:run
# 预期: 所有测试 passed

# 6. 前端服务正常
npm run dev
# 浏览器访问 http://localhost:5173

# == 联调验证 ==

# 7. 注册 → 登录 → 主界面 → 各页面导航
# (在浏览器中手动操作验证)

# 8. 后端控制台有请求日志输出
# (在操作前端时观察后端终端)
```

全部通过后，**Day 4 日志监控 + 测试结构 + 基础设施收尾完成**，第一阶段（项目架构搭建）圆满结束，可以进入 Day 5 数据与模型阶段。

文档版本：v1.0

适用项目：基于 YOLOv11 的目标检测智能体平台

最后更新：2026-06-30